

SlashRing — Solucoes v2

Cabinet ajustavel e funcional + Overdrive no nivel da especificacao. Documento de correcoes para levar o plugin ao nivel profissional/comercial.

Baseline assumido: todas as alteracoes da pasta `Maps` (documento de Solucoes anterior + redesign da UI) ja estao aplicadas no seu codigo local. O **overdrive** continua intacto (nao foi tocado por aquele documento).

Como aplicar: um item por vez, no seu editor. Depois de cada item: compile (Ctrl+F7 / F7), ouca e faca A-B em **volume igualado**, e faca `commit`. Nunca aplique tudo de uma vez. Se algo piorar: `git restore .` e volte ao ultimo commit bom.

Duas melhorias neste documento (como voce pediu)

- **Linhas longas quebradas:** nenhuma linha de codigo sai da pagina — as linhas grandes sao quebradas para a linha de baixo. As quebras que eu faco sao **C++ valido**, entao pode colar direto.
- **Mudancas pequenas com a linha exata:** alem do arquivo e da funcao, eu mostro o trecho a procurar (*Procure por*) e o que colocar no lugar. Onde o arquivo **nao** mudou desde o master (ex.: `OverdriveModule`) incluo o numero aproximado da linha; onde a pasta `Maps` ja mexeu (`PluginProcessor`, `PluginEditor`) a numeracao do seu arquivo local difere da do GitHub, entao eu ancorei pelo **trecho citado** — e so usar o Ctrl+F do editor.

Resumo das mudancas

- **P0 Cabinet ajustavel** — 4 parametros novos (Low Cut, High Cut, Level, Mix), bloco CABINET reescrito, `CabinetModule.h/.cpp` completos (dry/wet), knobs no editor. Resolve o cabinet matando o som e o deixa regulavel.
- **P1 Overdrive na spec** — tightness pre-clip (mid-forward TS) + controle de **Tone** novo, com knob no editor.
- **Diag IR de fabrica** — como testar e, se preciso, trocar o arquivo da IR.

Parte A — Cabinet: fazer soar e deixar ajustavel

A0 — Diagnostico (por que ainda mata o som)

Mesmo com o cabinet ja em modo IR-pura, o roteamento herdado ainda pode entregar o sinal para a **voz B, que nunca recebe a IR** (so a voz A recebe a IR de fabrica), e os valores de corte estavam **fixos no codigo** — voce nao tinha como ajustar por ouvido. O resultado e um som abafado, quase clean e baixo. A correcao: rotar 100% para a voz A (a que tem a IR), devolver o nivel a unidade com makeup, e expor **Low Cut, High Cut, Level e Mix** como parametros de verdade. Assim o cabinet passa a fazer o trabalho certo: cortar o fizz e entregar o timbre, com voce no controle.

A1 — Novos IDs de parametro

Mudanca pequena · Arquivo: `Source/PluginProcessor.h` · Funcao: `namespace ParameterID` · trecho perto de `cabinet_on`

PROCURE POR:

```
static constexpr auto cabinetOn = "cabinet_on";
```

ADICIONE LOGO ABAIXO:

```
static constexpr auto cabinetLowCut = "cabinet_low_cut";  
static constexpr auto cabinetHighCut = "cabinet_high_cut";  
static constexpr auto cabinetLevel = "cabinet_level";  
static constexpr auto cabinetMix = "cabinet_mix";
```

A2 — Criar os parametros

Mudanca pequena · Arquivo: Source/PluginProcessor.cpp · Funcao: `createParameterLayout()` · logo apos o Toggle Cabinet On

PROCURE POR:

```
parameters.push_back(std::make_unique<Toggle>(  
    ParameterID::cabinetOn,  
    "Cabinet On",  
    true));
```

ADICIONE LOGO ABAIXO:

```
parameters.push_back(std::make_unique<Parameter>(  
    ParameterID::cabinetLowCut,  
    "Cabinet Low Cut",  
    juce::NormalisableRange<float>(  
        20.0f, 300.0f, 1.0f, 0.5f),  
    80.0f));  
  
parameters.push_back(std::make_unique<Parameter>(  
    ParameterID::cabinetHighCut,  
    "Cabinet High Cut",  
    juce::NormalisableRange<float>(  
        2000.0f, 20000.0f, 1.0f, 0.5f),  
    12000.0f));  
  
parameters.push_back(std::make_unique<Parameter>(  
    ParameterID::cabinetLevel,  
    "Cabinet Level",  
    juce::NormalisableRange<float>(  
        -24.0f, 24.0f, 0.1f,  
        0.0f));  
  
parameters.push_back(std::make_unique<Parameter>(  
    ParameterID::cabinetMix,  
    "Cabinet Mix",  
    juce::NormalisableRange<float>(  
        0.0f, 1.0f, 0.01f,  
        1.0f));
```

O `0.5f` no fim das faixas de frequencia e o **skew** (resposta logaritmica), pra o knob girar de forma musical em Hz.

A3 — Reescrever o bloco CABINET (ler os parametros)

No `updateParameterState()`, localize o bloco `if (cabinetModule != nullptr) { ... }` (o que tinha os `constexpr` do cabinet) e **substitua o bloco inteiro** por:

```

if (cabinetModule != nullptr)
{
    const auto enabled =
        apvts.getRawParameterValue(
            ParameterID::cabinetOn)->load();

    const auto lowCut =
        apvts.getRawParameterValue(
            ParameterID::cabinetLowCut)->load();

    const auto highCut =
        apvts.getRawParameterValue(
            ParameterID::cabinetHighCut)->load();

    const auto levelDb =
        apvts.getRawParameterValue(
            ParameterID::cabinetLevel)->load();

    const auto mix =
        apvts.getRawParameterValue(
            ParameterID::cabinetMix)->load();

    cabinetModule->setEnabled(enabled > 0.5f);

    // Voz A carrega a IR de fabrica; voz B fica muda
    // (blend = 0). Low/High cut ajustam o timbre.
    cabinetModule->configureVoice(
        0, 0.0f, lowCut, highCut, 0.0f, 0.5f, 45.0f);
    cabinetModule->configureVoice(
        1, 0.0f, lowCut, highCut, 0.0f, 0.5f, 45.0f);

    // blend 0 = 100% voz A. 0 +6 dB compensa a lei de
    // pan (~0.5) do CabinetMixer, devolvendo o nivel a
    // unidade. levelDb e o makeup do usuario.
    cabinetModule->configureMixer(
        0.0f, 0.0f, 0.0f, levelDb + 6.0f);

    // Dry/wet do cabinet
    cabinetModule->setMix(mix);
}

```

Removi as chamadas `configurePhysics(...)`: no cabinet IR-pura elas viraram no-op. Pode remover sem medo (a IR ja captura alto-falante + caixa).

Ordem de build: aplique **A4 + A5** (que criam `setMix()`) **antes** de compilar o A3 — senao o compilador nao encontra `setMix`. O checklist no fim ja esta nessa ordem.

A4 — [arquivo completo](#) **Source/CabinetModule.h**

Adiciona o dry/wet (Mix) mantendo o resto da arquitetura. Substitua o arquivo inteiro:

```

#pragma once
#include "CabinetVoice.h"
#include "CabinetMixer.h"
#include "IRLoader.h"

class CabinetModule : public IRLoader::Listener
{
public:
    CabinetModule();
    ~CabinetModule() override;

```

```

void prepare (double sampleRate, int samplesPerBlock,
              int numChannels);
void prepare (const juce::dsp::ProcessSpec& spec);

void process (juce::AudioBuffer<float>& buffer) noexcept;
void process (juce::dsp::ProcessContextReplacing<float>&
              context) noexcept;

void reset();

void setEnabled (bool shouldBeEnabled) noexcept
    { isEnabled = shouldBeEnabled; }

// Dry/wet do cabinet (0 = seco, 1 = 100% com IR)
void setMix (float newMix) noexcept;

void triggerAsyncFactoryIRLoad (int voiceIndex,
                                const void* data,
                                size_t size,
                                const juce::String& uniqueHash);
void triggerAsyncUserIRLoad (int voiceIndex,
                              const juce::File& file);

void configureVoice (int voiceIndex, float gainDb,
                    float lowCut, float highCut,
                    float delayMs, float micDist,
                    float micAngle) noexcept;
void configurePhysics (int voiceIndex, float cabSize,
                      float openBack, float drive,
                      float breakup) noexcept;
void configureMixer (float blend, float pan,
                    float width, float outputDb) noexcept;

void irLoadingFinished (const juce::String& voiceId,
                        CachedIRData::Ptr irData) override;
void irLoadingFailed (const juce::String& voiceId,
                      const juce::String& reason) override;

private:
    bool isEnabled = true;

    CabinetVoice voiceA;
    CabinetVoice voiceB;
    CabinetMixer mixer;
    std::unique_ptr<IRLoader> irLoader;

    juce::AudioBuffer<float> localVoiceBufferA;
    juce::AudioBuffer<float> localVoiceBufferB;
    juce::dsp::AudioBlock<float> audioBlockA;
    juce::dsp::AudioBlock<float> audioBlockB;

    // Dry/wet – NOVO
    juce::AudioBuffer<float> dryBuffer;
    juce::SmoothedValue<float,
        juce::ValueSmoothingTypes::Linear> mixSmoothed;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (CabinetModule)
};

```

A5 — [arquivo completo](#) Source/CabinetModule.cpp

```

#include "CabinetModule.h"

CabinetModule::CabinetModule()
{
    irLoader = std::make_unique<IRLoader> (this);
}

```

```

}

CabinetModule::~CabinetModule() {}

void CabinetModule::prepare (double sampleRate,
                             int samplesPerBlock,
                             int numChannels)
{
    juce::dsp::ProcessSpec spec;
    spec.sampleRate = sampleRate;
    spec.maximumBlockSize =
        static_cast<juce::uint32> (samplesPerBlock);
    spec.numChannels =
        static_cast<juce::uint32> (numChannels);
    prepare (spec);
}

void CabinetModule::prepare (const juce::dsp::ProcessSpec& spec)
{
    irLoader->prepare (spec.sampleRate);

    voiceA.prepare (spec);
    voiceB.prepare (spec);
    mixer.prepare (spec);

    localVoiceBufferA.setSize (2, (int) spec.maximumBlockSize);
    localVoiceBufferB.setSize (2, (int) spec.maximumBlockSize);

    audioBlockA = juce::dsp::AudioBlock<float> (localVoiceBufferA);
    audioBlockB = juce::dsp::AudioBlock<float> (localVoiceBufferB);

    // Dry/wet
    dryBuffer.setSize ((int) spec.numChannels,
                      (int) spec.maximumBlockSize);
    mixSmoothed.reset (spec.sampleRate, 0.02);
    mixSmoothed.setCurrentAndTargetValue (1.0f);
}

void CabinetModule::process
    (juce::AudioBuffer<float>& buffer) noexcept
{
    if (! isEnabled)
        return;

    juce::ScopedNoDenormals noDenormals;

    const int numChannels = buffer.getNumChannels();
    const int numSamples = buffer.getNumSamples();

    // 1) Copia do sinal seco (antes do cabinet)
    for (int ch = 0; ch < numChannels; ++ch)
        dryBuffer.copyFrom (ch, 0, buffer, ch, 0, numSamples);

    // 2) Processa o molhado (IR) no proprio buffer
    juce::dsp::AudioBlock<float> block (buffer);
    juce::dsp::ProcessContextReplacing<float> ctx (block);
    process (ctx);

    // 3) Mistura dry/wet
    for (int s = 0; s < numSamples; ++s)
    {
        const float m = mixSmoothed.getNextValue();
        for (int ch = 0; ch < numChannels; ++ch)
        {
            float* wet = buffer.getWritePointer (ch);
            const float* dry = dryBuffer.getReadPointer (ch);
            wet[s] = dry[s] * (1.0f - m) + wet[s] * m;
        }
    }
}

```

```

    }
}

void CabinetModule::process
(juce::dsp::ProcessContextReplacing<float>& context) noexcept
{
    if (! isEnabled || context.isBypassed)
        return;

    auto inBlock = context.getInputBlock();
    auto outBlock = context.getOutputBlock();
    size_t samples = inBlock.getNumSamples();

    auto activeBlockA = audioBlockA.getSubBlock (0, samples);
    auto activeBlockB = audioBlockB.getSubBlock (0, samples);

    activeBlockA.copyFrom (inBlock);
    activeBlockB.copyFrom (inBlock);

    juce::dsp::ProcessContextReplacing<float> ctxA (activeBlockA);
    juce::dsp::ProcessContextReplacing<float> ctxB (activeBlockB);

    voiceA.process (ctxA);
    voiceB.process (ctxB);

    mixer.process (activeBlockA, activeBlockB, outBlock);
}

void CabinetModule::setMix (float newMix) noexcept
{
    mixSmoothed.setTargetValue (juce::jlimit (0.0f, 1.0f, newMix));
}

void CabinetModule::triggerAsyncFactoryIRLoad
(int voiceIndex, const void* data, size_t size,
 const juce::String& uniqueHash)
{
    juce::String id = (voiceIndex == 0) ? "A" : "B";
    irLoader->submitLoadTask (id, data, size, uniqueHash);
}

void CabinetModule::triggerAsyncUserIRLoad
(int voiceIndex, const juce::File& file)
{
    juce::String id = (voiceIndex == 0) ? "A" : "B";
    irLoader->submitLoadTask (id, file);
}

void CabinetModule::configureVoice
(int voiceIndex, float gainDb, float lowCut, float highCut,
 float delayMs, float micDist, float micAngle) noexcept
{
    if (voiceIndex == 0)
        voiceA.setVoiceParameters (gainDb, lowCut, highCut,
                                   delayMs, micDist, micAngle);
    else
        voiceB.setVoiceParameters (gainDb, lowCut, highCut,
                                   delayMs, micDist, micAngle);
}

void CabinetModule::configurePhysics
(int voiceIndex, float cabSize, float openBack,
 float drive, float breakup) noexcept
{
    if (voiceIndex == 0)
        voiceA.setPhysics (cabSize, openBack, drive, breakup);
}

```

```

        else
            voiceB.setPhysics (cabSize, openBack, drive, breakup);
    }

    void CabinetModule::configureMixer
        (float blend, float pan, float width, float outputDb) noexcept
    {
        mixer.setMixerParams (blend, pan, width, outputDb);
    }

    void CabinetModule::irLoadingFinished
        (const juce::String& voiceId, CachedIRData::Ptr irData)
    {
        if (voiceId == "A")
            voiceA.updateIRBuffer (irData);
        else if (voiceId == "B")
            voiceB.updateIRBuffer (irData);
    }

    void CabinetModule::irLoadingFailed
        (const juce::String& voiceId, const juce::String& reason)
    {
        juce::Logger::writeToLog (
            "--- SLASHRING DSP ERROR --- Voice ID: "
            + voiceId + " -> " + reason);
    }

    void CabinetModule::reset()
    {
        voiceA.reset();
        voiceB.reset();
        mixer.reset();
        dryBuffer.clear();
        mixSmoothed.setCurrentAndTargetValue (
            mixSmoothed.getTargetValue());
    }

```

A6 — CabinetVoice sem mudança

O `CabinetVoice.h/.cpp` já é a versão IR-pura da pasta `Maps` — ele já aceita Low Cut / High Cut / gain via `setVoiceParameters(...)`, que é exatamente o que o bloco A3 alimenta. Não precisa tocar nele. (Se por algum motivo não estiver IR-pura, use a versão completa do documento de Soluções anterior.)

A7 — Editor: declarar os knobs do cabinet

Mudança pequena · Arquivo: `Source/PluginEditor.h` · Função: `class ...Editor` (membros) · onde estão os controles do cabinet

PROCURE POR:

```
juce::ToggleButton cabinetButton { "CABINET" };
juce::Label cabinetInfo;
```

TROQUE POR (TROCA O LABEL POR 4 KNOBS):

```
juce::ToggleButton cabinetButton { "CABINET" };
Knob cabLowCut { "LOW CUT" };
Knob cabHighCut { "HIGH CUT" };
Knob cabLevel { "LEVEL" };
Knob cabMix { "MIX" };
```

Mudanca pequena · Arquivo: Source/PluginEditor.h · Funcao: attachments · junto dos outros SliderAtt

PROCURE POR:

```
std::unique_ptr<SliderAtt> delayTimeAtt, delayFbAtt,  
                           delayMixAtt, reverbMixAtt;
```

ADICIONE ABAIXO:

```
std::unique_ptr<SliderAtt> cabLowCutAtt, cabHighCutAtt,  
                           cabLevelAtt, cabMixAtt;
```

A8 — Editor: criar, ligar e posicionar os knobs

Mudanca pequena · Arquivo: Source/PluginEditor.cpp · Funcao: construtor do editor · o bloco do cabinetInfo

PROCURE POR:

```
cabinetInfo.setText ("IR-BASED", juce::dontSendNotification);  
cabinetInfo.setJustificationType (juce::Justification::centred);  
cabinetInfo.setColour (juce::Label::textColourId,  
                       juce::Colour (0xff9A9188));  
addAndMakeVisible (cabinetInfo);
```

REMOVA ESSE BLOCO INTEIRO (O LABEL SOME; ENTRAM OS KNOBS).

```
// (removido – cabinetInfo substituido pelos knobs)
```

Ainda no construtor, no `for` que faz `addAndMakeVisible` dos knobs, **inclua na lista** os quatro nomes novos: `&cabLowCut`, `&cabHighCut`, `&cabLevel`, `&cabMix`. E confira que **nenhuma** outra linha ainda cita `cabinetInfo`.

Mudanca pequena · Arquivo: Source/PluginEditor.cpp · Funcao: construtor (attachments) · logo apos o attachment do reverbMix

PROCURE POR:

```
reverbMixAtt = std::make_unique<SliderAtt> (  
    s, ParameterID::reverbMix, reverbMix.slider);
```

ADICIONE ABAIXO:

```
cabLowCutAtt = std::make_unique<SliderAtt> (  
    s, ParameterID::cabinetLowCut, cabLowCut.slider);  
cabHighCutAtt = std::make_unique<SliderAtt> (  
    s, ParameterID::cabinetHighCut, cabHighCut.slider);  
cabLevelAtt = std::make_unique<SliderAtt> (  
    s, ParameterID::cabinetLevel, cabLevel.slider);  
cabMixAtt = std::make_unique<SliderAtt> (  
    s, ParameterID::cabinetMix, cabMix.slider);
```

Mudanca pequena · Arquivo: Source/PluginEditor.cpp · Funcao: `resized()` – divisao da linha B · onde a linha B e fatiada

PROCURE POR:


```
auto cabArea = rowB.removeFromLeft (180); rowB.removeFromLeft (gap);
auto dlyArea = rowB.removeFromLeft (360); rowB.removeFromLeft (gap);
```

TROQUE POR (DA MAIS ESPACO AO CABINET):

```
auto cabArea = rowB.removeFromLeft (240); rowB.removeFromLeft (gap);
auto dlyArea = rowB.removeFromLeft (300); rowB.removeFromLeft (gap);
```

Mudanca pequena · Arquivo: Source/PluginEditor.cpp · Funcao: `resized()` — bloco CABINET · o bloco que posicionava toggle + info

PROCURE POR:

```
// CABINET: toggle + info
{
    auto r = inner (cabArea);
    cabinetButton.setBounds (r.removeFromTop (28));
    r.removeFromTop (8);
    cabinetInfo.setBounds (r.removeFromTop (24));
}
```

TROQUE POR (TOGGLE + 4 KNOBS EM GRADE 2X2):

```
// CABINET: toggle + 4 knobs (2x2)
{
    auto r = inner (cabArea);
    cabinetButton.setBounds (r.removeFromTop (28));
    r.removeFromTop (6);
    auto top = r.removeFromTop (r.getHeight() / 2);
    auto bot = r;
    int kwT = top.getWidth() / 2;
    cabLowCut .setBounds (top.removeFromLeft (kwT).reduced (2));
    cabHighCut.setBounds (top.reduced (2));
    int kwB = bot.getWidth() / 2;
    cabLevel .setBounds (bot.removeFromLeft (kwB).reduced (2));
    cabMix .setBounds (bot.reduced (2));
}
```

A9 — Se ainda soar escuro: e a IR

Depois de A1–A8, o roteamento e o nivel estao corretos e tudo e ajustavel. Se o timbre **ainda** vier abafado mesmo com o **High Cut** aberto (12 kHz) e o **Level** igualado, o problema e o **arquivo da IR de fabrica** (`Marshall_Creamback.wav`), nao o codigo.

Lembre-se: a funcao do cabinet e cortar o fizz — algum escurecimento e correto e desejavel (filtrar o ruido). O alvo e um rolloff musical em torno de ~5 kHz, nao um abafamento em 2 kHz.

Teste rapido (A/B): carregue a mesma IR no Ignite Amps NadIR (IR loader gratis) e compare em volume igualado. Se soar diferente, o problema e o seu processamento (nivel/resample/trim). Se soar **igual e escuro**, a IR e escura — troque o arquivo por uma IR de **Marshall 1960 (4x12) + Celestion V30, microfonada com SM57** perto do cone, levemente fora de eixo. O timbre vem da IR; o codigo so entrega.

Parte B — Overdrive no nível da especificação

B0 — Diagnostico vs. spec (TS/SD-1)

O overdrive atual (revisão TD-010) é um boost de **um estágio**: HPF de entrada em 80 Hz, um único soft-clip assimétrico (tanh fixo 1.35/1.05), LPF de saída fixo em 8500 Hz e drive 0–30 dB. Ele já está limpo e estável — falta o **caráter** da spec. Comparado a um TS/SD-1 faltam dois pontos: **(1)** o aperto mid-forward que vem de um HPF pre-clip mais alto (tira o grave antes de clipar, deixa apertado e com médio presente); e **(2)** um controle de **Tone**. Vamos adicionar os dois. O clip assimétrico já bate com o caráter SD-1, então fica.

B1 — Overdrive: declarar Tone

Mudança pequena · Arquivo: Source/OverdriveModule.h · Função: `class OverdriveModule` · linha aprox. 44 (após `setLevel`)

PROCURE POR:

```
void setLevel(float newLevel);
```

ADICIONE ABAIXO:

```
void setTone(float newTone);
```

Mudança pequena · Arquivo: Source/OverdriveModule.h · Função: `seção STATE` (membros privados) · linha aprox. 108 (após `currentSampleRate`)

PROCURE POR:

```
double currentSampleRate = 44100.0;
```

ADICIONE ABAIXO:

```
// Tone (LPF de saída controlável) — NOVO
float toneHz = 5600.0f;
```

B2 — Overdrive: tightness + Tone

Mudança pequena · Arquivo: Source/OverdriveModule.cpp · Função: `prepare()` · linha aprox. 74 (HPF de entrada)

PROCURE POR:

```
*inputHPF.state =
    *juce::dsp::IIR::Coefficients<float>::makeHighPass(
        sampleRate,
        80.0f);
```

TROQUE POR (APERTA O GRAVE ANTES DO CLIP — CARÁTER TS):

```
*inputHPF.state =
    *juce::dsp::IIR::Coefficients<float>::makeHighPass(
        sampleRate,
        250.0f);
```

Mudanca pequena · Arquivo: `Source/OverdriveModule.cpp` · Funcao: `prepare()` · linha aprox. 80 (LPF de saida)

PROCURE POR:

```
*outputLPF.state =
    *juce::dsp::IIR::Coefficients<float>::makeLowPass(
        sampleRate,
        8500.0f);
```

TROQUE POR (O LPF DE SAIDA PASSA A SER O TONE):

```
*outputLPF.state =
    *juce::dsp::IIR::Coefficients<float>::makeLowPass(
        sampleRate,
        toneHz);
```

E adicione a funcao `setTone` (ex.: logo apos `OverdriveModule::setLevel(...)`):

```
void OverdriveModule::setTone(
    float newTone)
{
    const float t =
        juce::jlimit(0.0f, 10.0f, newTone);

    // 0..10 -> 2 kHz .. 8 kHz
    const float hz =
        juce::jmap(t, 0.0f, 10.0f, 2000.0f, 8000.0f);

    // So recomputa se mudou (evita alocao por bloco)
    if (std::abs(hz - toneHz) < 0.5f)
        return;

    toneHz = hz;

    *outputLPF.state =
        *juce::dsp::IIR::Coefficients<float>::makeLowPass(
            currentSampleRate,
            toneHz);
}
```

Quer um TS mais fino/classico? Suba o HPF pre-clip de `250.0f` para algo entre 400–700 Hz. 250 Hz e um meio-termo apertado e musical.

B3 — Parametro Tone: ID

Mudanca pequena · Arquivo: `Source/PluginProcessor.h` · Funcao: `namespace ParameterID` · perto de `overdrive_level`

PROCURE POR:

```
static constexpr auto overdriveLevel = "overdrive_level";
```

ADICIONE ABAIXO:

```
static constexpr auto overdriveTone = "overdrive_tone";
```

B4 — Parametro Tone: criar

Mudanca pequena · Arquivo: Source/PluginProcessor.cpp · Funcao: createParameterLayout() · logo apos o parametro Overdrive Level

PROCURE POR:

```
parameters.push_back(std::make_unique<Parameter>(
    ParameterID::overdriveLevel,
    "Overdrive Level",
    juce::NormalisableRange<float>(
        -60.0f,
        12.0f,
        0.01f),
    0.0f));
```

ADICIONE ABAIXO:

```
parameters.push_back(std::make_unique<Parameter>(
    ParameterID::overdriveTone,
    "Overdrive Tone",
    juce::NormalisableRange<float>(
        0.0f, 10.0f, 0.01f),
    6.0f));
```

B5 — Ligar o Tone no processamento

Mudanca pequena · Arquivo: Source/PluginProcessor.cpp · Funcao: updateParameterState() – bloco OVERDRIVE · onde le overdriveLevel

PROCURE POR:

```
const auto overdriveLevel =
    apvts.getRawParameterValue(
        ParameterID::overdriveLevel)->load();
```

ADICIONE ABAIXO:

```
const auto overdriveTone =
    apvts.getRawParameterValue(
        ParameterID::overdriveTone)->load();
```

Mudanca pequena · Arquivo: Source/PluginProcessor.cpp · Funcao: updateParameterState() – bloco OVERDRIVE · ainda dentro do if (overdriveModule != nullptr), apos o else

PROCURE POR:

```
else
{
    overdriveModule->setDrive(0.0f);
    overdriveModule->setLevel(0.0f);
}
```

ADICIONE ABAIXO (O TONE VALE LIGADO OU DESLIGADO):

```
overdriveModule->setTone(overdriveTone);
```

B6 — Editor: knob de Tone

Mudanca pequena · Arquivo: Source/PluginEditor.h · Funcao: membros do editor · onde estao odDrive/odLevel

PROCURE POR:

```
Knob odDrive { "DRIVE" };
Knob odLevel { "LEVEL" };
```

TROQUE POR:

```
Knob odDrive { "DRIVE" };
Knob odLevel { "LEVEL" };
Knob odTone { "TONE" };
```

Mudanca pequena · Arquivo: Source/PluginEditor.h · Funcao: attachments · onde estao odDriveAtt/odLevelAtt

PROCURE POR:

```
std::unique_ptr<SliderAtt> odDriveAtt, odLevelAtt;
```

TROQUE POR:

```
std::unique_ptr<SliderAtt> odDriveAtt, odLevelAtt, odToneAtt;
```

No construtor: no for do addAndMakeVisible, **inclua** &odTone na lista; e adicione o attachment:

```
odToneAtt = std::make_unique<SliderAtt> (
    s, ParameterID::overdriveTone, odTone.slider);
```

Mudanca pequena · Arquivo: Source/PluginEditor.cpp · Funcao: resized() — divisao da linha A · onde a linha A e fatiada

PROCURE POR:

```
auto odArea = rowA.removeFromLeft (220); rowA.removeFromLeft (gap);
```

TROQUE POR (ESPACO P/ 3 KNOBS):

```
auto odArea = rowA.removeFromLeft (300); rowA.removeFromLeft (gap);
```

Mudanca pequena · Arquivo: Source/PluginEditor.cpp · Funcao: resized() — bloco OVERDRIVE · onde posiciona os 2 knobs

PROCURE POR:

```
int kw = r.getWidth() / 2;
odDrive.setBounds (r.removeFromLeft (kw).reduced (2));
odLevel.setBounds (r.reduced (2));
```

TROQUE POR (3 KNOBS):

```
int kw = r.getWidth() / 3;
odDrive.setBounds (r.removeFromLeft (kw).reduced (2));
```

```
odLevel.setBounds (r.removeFromLeft (kw).reduced (2));  
odTone .setBounds (r.reduced (2));
```

Ordem de aplicacao (checklist)

- **1.** A1 + A2 — parametros do cabinet → build.
- **2.** A4 + A5 — `CabinetModule.h/.cpp` (dry/wet; cria setMix) → build → commit.
- **3.** A3 — bloco CABINET (usa setMix) → build → gire Low/High/Level/Mix e ouca → commit.
- **4.** A7 + A8 — knobs do cabinet no editor → build → confira os 4 knobs → commit.
- **5.** A9 — se ainda escuro, A/B da IR e troque o arquivo se preciso.
- **6.** B1 + B2 — tightness + Tone no OverdriveModule → build → A/B → commit.
- **7.** B3 + B4 + B5 — parametro Tone + wiring → build → commit.
- **8.** B6 — knob Tone no editor → build → commit.

Regra de ouro: um item, um build, um commit. Se algo piorar, `git restore` e volte ao ultimo commit bom.

Pontos de partida (timbre Slash)

- **Cabinet:** Low Cut ~80 Hz · High Cut ~10-12 kHz (baixe se tiver fizz) · Level a gosto (igualar com bypass) · Mix 100%.
- **Overdrive:** Drive ~2-3 · Tone ~5-6 · Level ~7-8 (e boost — empurra o amp).